

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 07 · LECTURE NOTES

Newton and Interior Point

Using curvature, and why linear programming is fast.

MODULE OUTCOMES

- Explain Newton's method and its convergence.
- Explain why Newton is affine invariant and what that means.
- Explain quasi-Newton methods and when they are the right choice.
- Explain the barrier method and the central path.
- Explain why interior point methods transformed linear programming.

1 Newton's method

$$\mathbf{x} \leftarrow \mathbf{x} - [\nabla^2 f(\mathbf{x})]^{-1} \nabla f(\mathbf{x})$$

Build the second-order Taylor model of f at the current point and jump to that quadratic's exact minimum. Near a non-degenerate optimum this converges **quadratically** — **the number of correct digits roughly doubles each iteration**, so six iterations can achieve what thousands of gradient steps would. **The cost is $O(n^3)$ per step** to solve the Newton system (less for sparse or structured Hessians), and that cost is the entire obstacle at scale — which §2 addresses.

NEWTON IS AFFINE INVARIANT AND GRADIENT DESCENT IS NOT

Rescale or linearly transform the variables, and Newton's iterates follow the same path under the same transformation. Its behaviour does not depend on the coordinate system at all.

Gradient descent is not affine invariant. Its path depends entirely on the scaling of the variables — **which is precisely why feature scaling matters so much** (Module 05 §1, CSCE 633 Module 02 §2). The method's notion of 'steepest' is a statement about the coordinates, not about the function.

So Newton is immune to the condition number, in the sense that κ does not appear in its local convergence rate — a badly scaled problem and a well scaled one take the same number of Newton steps.

That is the real content of the claim that 'Newton uses curvature'. It is not merely a faster first-order method; it is a method whose notion of distance is derived from the problem's own geometry rather than from an arbitrary choice of coordinates. And it explains why preconditioning a first-order method is, in effect, an attempt to approximate this property cheaply.

2 Making second order affordable

Method	What it uses	Cost per step
Gradient descent	First derivatives only.	$O(n)$, and $O(\kappa)$ iterations.
L-BFGS	An implicit Hessian approximation built from the last m gradient differences.	$O(mn)$ with m around 10. The practical default for smooth unconstrained problems, and consistently under-used in machine learning.
BFGS	A full dense Hessian approximation, updated each step.	$O(n^2)$ storage, which caps the problem size — hence the limited-memory variant.
Newton	The exact Hessian.	$O(n^3)$ for a dense solve, far less if the Hessian is sparse or structured.
Newton-CG (truncated Newton)	Hessian-vector products only — the Newton system is solved approximately by conjugate gradients.	Never forms or stores the Hessian, so it scales to very large n — see the callout.
Gauss-Newton	The Hessian approximated from the Jacobian, valid for least squares with small residuals.	The basis of Levenberg–Marquardt, and the standard method for nonlinear least squares — including bundle adjustment (CSCE 748).

YOU NEVER NEED THE HESSIAN, ONLY ITS ACTION

Newton's step requires solving $\nabla^2 f \cdot d = -\nabla f$, and an iterative linear solver — conjugate gradients — requires only the ability to compute $\nabla^2 f \cdot v$ for given vectors v . **It never needs the matrix itself.**

And a Hessian-vector product costs roughly one extra gradient evaluation. It is the directional derivative of the gradient, which automatic differentiation computes directly (CSCE 636 Module 02) — a forward-over-reverse or reverse-over-reverse composition, available in every major framework.

So second-order methods scale to problems where the Hessian could never be stored, let alone factorised — a million-parameter model has a Hessian with 10^{12} entries and perfectly usable Hessian-vector products.

This is why Newton-CG and trust-region methods are viable on large problems, and why dismissing second-order methods as ' $O(n^3)$, therefore impractical' is out of date — a dismissal that nonetheless remains common, and which the Hessian-free optimisation literature addressed directly.

3 Interior point methods

Replace the constraints with a BARRIER in the objective:

```
minimise  t*f(x) - sum_i log(-g_i(x))
```

The log barrier is $+\infty$ at the boundary, so every iterate stays strictly feasible. The result is smooth -- solve it with Newton.

THE CENTRAL PATH: the solution $x^*(t)$ as t varies. Small t : the barrier dominates and x^* sits near the analytic centre. Large t : the objective dominates and $x^*(t)$ approaches the constrained optimum.

S0: solve for an increasing sequence of t , WARM STARTING each solve from the previous solution. Each takes a few Newton steps because it begins close.

THE GUARANTEE

the duality gap at $x^*(t)$ is EXACTLY m/t , with m the number of inequality constraints. You always KNOW how suboptimal you are -- Module 04's certificate, continuously available, free.

Total: $O(\sqrt{m})$ outer iterations. POLYNOMIAL.

The explicit m/t duality gap is the most elegant thing in this module. It converts the barrier method from a plausible heuristic into an algorithm with a guarantee at every iterate — you can stop at any point and state exactly how far from optimal you are, which is Module 04's certificate delivered continuously and without extra work.

WHY INTERIOR POINT CHANGED LINEAR PROGRAMMING

Khachiyan's ellipsoid method (1979) proved that linear programming is solvable in polynomial time — a major theoretical result — **and was entirely useless in practice**, being slower than simplex on every real instance.

Karmarkar's interior point method (1984) was polynomial and competitive — the first algorithm that was both, and it caused a genuine upheaval, including a patent dispute and a great deal of commercial interest.

The competition improved both. Simplex implementations got dramatically faster through the late 1980s and 1990s in direct response, with better pivoting, better factorisation updates, and much better presolve — **so the net effect was far larger than either method alone**, and solvers now carry both and select per instance.

Interior point wins on large sparse problems; simplex wins when a warm start is available — **which is why branch and bound uses simplex for its thousands of nearly-identical re-solves** (Module 09 §2), since interior point cannot warm start effectively. The two are complements rather than competitors, which was not obvious at the time.

4 Choosing

- **Smooth, unconstrained, n up to roughly 10^4 : L-BFGS.** Few hyperparameters, fast convergence, and a well-tested implementation in every scientific library.
- **Constrained convex: interior point**, reached through a modelling layer such as CVXPY so that you write the problem and not the algorithm.
- **Very large, stochastic, non-convex: first-order methods. Curvature estimates from mini-batches are noisy, and the Hessian of a neural network changes faster than it can be usefully estimated** — which is the good reason second-order methods have not displaced SGD in deep learning, as distinct from the habitual one.
- **Nonlinear least squares: Gauss–Newton or Levenberg–Marquardt**, which exploit the residual structure to approximate the Hessian from the Jacobian — and which underlie bundle adjustment and camera calibration (CSCE 748).
- **And second-order methods are genuinely under-used in machine learning** — partly for the good reason above, and partly from habit: **a deterministic full-batch problem of moderate size should usually be solved with L-BFGS rather than with Adam**, and frequently is not.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Boyd & Vandenberghe — Convex Optimization, chapters 9, 10 and 11 (free)	Newton, equality-constrained Newton, and the barrier method with the m/t result — the core of this module.
Nocedal & Wright — Numerical Optimization, chapters 6, 7 and 19	Quasi-Newton methods and the practical detail of §2, including L-BFGS's two-loop recursion.
Martens — Deep Learning via Hessian-Free Optimization (free)	The §2 callout applied to neural networks — Hessian-vector products via automatic differentiation, at scale.
Wright — Primal-Dual Interior-Point Methods	The reference for §3's method as actually implemented in solvers.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement Newton's method** and verify quadratic convergence by plotting the error on a log-log scale.

- 2 **Demonstrate affine invariance:** rescale the variables and show Newton's iteration count is unchanged while gradient descent's is not.
- 3 Implement L-BFGS, or use one, and compare against gradient descent and Newton on the same problem.
- 4 **Compute a Hessian-vector product by automatic differentiation** and verify it against the explicit Hessian.
- 5 **Implement Newton-CG** using only Hessian-vector products.
- 6 Implement the log barrier method for a small linear program.
- 7 **Trace the central path** by solving for a sequence of t , and plot it.
- 8 **Verify the duality gap is m/t** at each point on the path.
- 9 Compare simplex and interior point on a large sparse LP and on a small dense one.
- 10 **Solve a full-batch problem from CSCE 633 with L-BFGS** and compare against Adam.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State Newton's step and its convergence rate.
- 2 What does affine invariance mean, and why does it matter?
- 3 Compare six methods on what they use and what they cost.
- 4 Why do you never need the Hessian itself?
- 5 Describe the barrier method and the central path.
- 6 What is the duality gap on the central path, and why does that matter?
- 7 Why did interior point change linear programming, and what does simplex still win at?
- 8 Give four situations and the method for each.